PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

SELECTED 10 CASE STUDIES — REPORT

Student Name	N. Guna Varshini
Roll Number	26001407
Branch / Section	CSD-1 (Sec-9)
Course	Python Programming
Academic Year	2026–2027

Each case study includes a problem statement, objectives, assumptions, algorithm, pseudocode, Python source code, sample output, test cases, flow sequence, and conclusion.

CONTENTS

Selected case studies from the supplied Python Activity-Based Learning list:

Case Study 2 — Progressive Electricity Billing

Case Study 13 — ATM Transaction Sequence Simulation

Case Study 24 — Courier Charge Optimization

Case Study 31 — String-Based Examination Answer Evaluation

Case Study 47 — Sports Tournament Ranking System

Case Study 33 — Campus Registration Identifier Analyzer

Case Study 64 — Tuple-Based Course Registration Validation

Case Study 75 — Integrated Student Record Consistency Checker

Case Study 89 — University Hostel Room Allocation

Case Study 92 — Multi-Event Sports Scheduling and Scoring

Project Objectives

- Understand practical problems using Python.
- Develop logical and computational thinking.
- Represent solutions using algorithms, pseudocode, and flow sequences.
- Implement and test Python programs using the assigned syllabus.
- Analyse program outputs and draw conclusions.

CASE STUDY 2 — Progressive Electricity Billing

Question / Problem Statement

Calculate household electricity bills using consumption slabs, fixed charges, surcharges, and rebates. Compare bills across months and identify an unusual increase.

Objectives

- Calculate energy charges using progressive slabs.
- Add fixed charges and a surcharge, then apply a qualifying rebate.
- Compare monthly bills and flag a large increase.

Assumptions

- First 100 units cost Rs. 2.00 per unit.
- Next 100 units cost Rs. 3.00 per unit; units above 200 cost Rs. 5.00 per unit.
- Fixed charge is Rs. 50; surcharge is 5% of energy charges; rebate is Rs. 25 when consumption is at most 100 units.
- An increase above 30% from the previous month is flagged.

Algorithm

1. Read each month's consumption.
2. Calculate energy charges progressively across the three slabs.
3. Add fixed charge and surcharge; subtract an eligible rebate.
4. Compare the current bill with the previous bill and print the result.

Pseudocode

```
START
Set monthly unit readings and billing rules
FOR each month: calculate slab charges
Add fixed charge and surcharge; apply rebate if eligible
Compare bill with previous month
Display bill and increase warning if above 30%
END
```

Python Program

```
def energy_charge(units):
    if units <= 100:
        return units * 2
    elif units <= 200:
        return 100 * 2 + (units - 100) * 3
    return 100 * 2 + 100 * 3 + (units - 200) * 5

monthly_units = [80, 130, 220]
previous_bill = None

for month, units in enumerate(monthly_units, start=1):
    energy = energy_charge(units)
    surcharge = energy * 0.05
    rebate = 25 if units <= 100 else 0
    bill = energy + 50 + surcharge - rebate
    print("Month", month, "- units:", units,
          "bill: Rs.", round(bill, 2))
    if previous_bill is not None:
        increase = (bill - previous_bill) / previous_bill * 100
        if increase > 30:
            print("Warning: bill increased by",
                  round(increase, 2), "%")
    previous_bill = bill
```

Sample Output



Test Cases

Input / Condition	Expected Result
80 units	Rs. 193.00; rebate applied

Input / Condition	Expected Result
130 units	Rs. 365.00; no rebate
220 units	Rs. 785.00; high-use slab applied

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The program calculates progressive electricity charges, applies the stated fees and rebate, and highlights large month-to-month bill increases.

CASE STUDY 13 — ATM Transaction Sequence Simulation

Question / Problem Statement

Process deposits and withdrawals in sequence, maintain an account balance, reject invalid transactions, and summarize accepted and rejected operations.

Objectives

- Update the balance after each valid transaction.
- Reject negative amounts and withdrawals exceeding the balance.
- Display transaction outcomes and the final balance.

Assumptions

- Opening balance is Rs. 5,000.
- Transactions are supplied as (type, amount) tuples.
- Only positive amounts are accepted; withdrawals cannot exceed the current balance.

Algorithm

1. Set the opening balance and transaction list.
2. Process each transaction in order.
3. Validate amount and transaction type.
4. Apply valid deposits or withdrawals and print the balance.
5. Display the final balance and transaction counts.

Pseudocode

```
START
Set opening balance and transactions
FOR each transaction: validate amount and type
IF deposit: add amount; ELSE IF valid withdrawal: subtract amount
Otherwise reject transaction
Display transaction result and final summary
END
```

Python Program

```
balance = 5000
transactions = [
    ("deposit", 1500),
    ("withdraw", 2000),
    ("withdraw", 6000),
    ("deposit", -100),
    ("withdraw", 500)
]
accepted = 0
rejected = 0

for kind, amount in transactions:
    if amount <= 0:
        print(kind, amount, "- rejected: invalid amount")
        rejected += 1
    elif kind == "deposit":
        balance += amount
        accepted += 1
        print("Deposit accepted; balance:", balance)
    elif kind == "withdraw":
        if amount <= balance:
            balance -= amount
            accepted += 1
            print("Withdrawal accepted; balance:", balance)
        else:
            print("Withdrawal rejected: insufficient balance")
            rejected += 1
    else:
        print("Rejected: unknown transaction type")
        rejected += 1

print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
print("Final balance: Rs.", balance)
```

Sample Output

Python Console

```
Deposit accepted; balance: 6500
Withdrawal accepted; balance: 4500
Withdrawal rejected: insufficient balance
deposit -100 - rejected: invalid amount
Withdrawal accepted; balance: 4000
Accepted transactions: 3
Rejected transactions: 2
Final balance: Rs. 4000
```

```
deposit -100 - rejected: invalid amount
Withdrawal accepted; balance: 4000
Accepted transactions: 3
Rejected transactions: 2
Final balance: Rs. 4000
```

Test Cases

Input / Condition	Expected Result
Deposit Rs. 1,500	Accepted; balance Rs. 6,500
Withdraw Rs. 6,000	Rejected; insufficient balance
Deposit -100	Rejected; invalid amount

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The simulation maintains an accurate balance and safely rejects invalid or unaffordable transactions.

CASE STUDY 24 — Courier Charge Optimization

Question / Problem Statement

Calculate delivery charges from parcel weight, distance zone, delivery speed, and declared handling needs. Compare available service options and select the least expensive eligible option.

Objectives

- Calculate charges for several courier services.
- Apply urgency and handling surcharges.
- Choose the lowest-cost eligible service.

Assumptions

- Base charge is Rs. 40 per parcel plus Rs. 15 per kg.
- Zone multipliers are Local 1.0, Regional 1.5, National 2.0.
- Express delivery adds Rs. 50; fragile handling adds Rs. 30.
- All listed services can handle the sample parcel.

Algorithm

1. Store parcel details and available services.
2. Calculate weight and zone charges for each service.
3. Add applicable express and fragile fees.
4. Print each quote and track the cheapest service.
5. Display the selected service and its cost.

Pseudocode

```
START
Set parcel weight, zone, urgency and handling
FOR each eligible service: calculate base and surcharges
Print quote and compare with cheapest quote
Display cheapest service and cost
END
```

Python Program

```
weight = 3.0
zone = "National"
express = True
fragile = True
zone_rates = {"Local": 1.0, "Regional": 1.5, "National": 2.0}
services = [("QuickShip", 40, 15),
            ("ValueCourier", 30, 12),
            ("SafeParcel", 45, 10)]
best_service = ""
best_cost = None

for name, base, per_kg in services:
    cost = (base + weight * per_kg) * zone_rates[zone]
    if express:
        cost += 50
    if fragile:
        cost += 30
    print(name, "- charge: Rs.", round(cost, 2))
    if best_cost is None or cost < best_cost:
        best_cost = cost
        best_service = name

print("Cheapest service:", best_service)
print("Minimum charge: Rs.", round(best_cost, 2))
```

Sample Output



Test Cases

Input / Condition	Expected Result
National, 3 kg, express, fragile	ValueCourier: Rs. 226
Local zone	Zone multiplier becomes 1.0
Standard delivery	Express fee is not added

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The program compares courier quotes consistently and selects the least expensive service for the supplied parcel requirements.

CASE STUDY 31 — String-Based Examination Answer Evaluation

Question / Problem Statement

Compare student answer strings with an answer key, count correct, incorrect, and unanswered responses, apply negative marking, and calculate the final score.

Objectives

- Compare answers with the answer key.
- Count correct, incorrect, and blank answers.
- Calculate marks after negative marking.

Assumptions

- Each correct answer earns 4 marks.
- Each incorrect answer loses 1 mark.
- A blank answer earns 0 marks.
- Comparison ignores letter case and surrounding spaces.

Algorithm

1. Store the answer key and student answers.
2. Compare each answer after normalization.
3. Update correct, incorrect, and unanswered counts.
4. Calculate the score using the marking scheme.
5. Display the result summary.

Pseudocode

```
START
Set answer key and student answers
FOR each question: normalize both strings
IF equal and nonblank: correct; ELSE IF blank: unanswered; ELSE: incorrect
Calculate score = correct × 4 − incorrect
Display counts and score
END
```

Python Program

```
answer_key = ["A", "C", "B", "D", "A"]
answers = ["a", "B", "", "D", "A"]
correct = 0
incorrect = 0
unanswered = 0

for key, answer in zip(answer_key, answers):
    key = key.strip().upper()
    answer = answer.strip().upper()
    if answer == "":
        unanswered += 1
    elif answer == key:
        correct += 1
    else:
        incorrect += 1

score = correct * 4 - incorrect
print("Correct:", correct)
print("Incorrect:", incorrect)
print("Unanswered:", unanswered)
print("Final score:", score, "out of", len(answer_key) * 4)
```

Sample Output



Test Cases

Input / Condition	Expected Result
Answer matches key	Add 4 marks

Input / Condition	Expected Result
Answer is different	Subtract 1 mark
Empty answer	Count unanswered; add 0 marks

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The evaluator uses string normalization to compare answers and calculates a score according to the stated marking policy.

CASE STUDY 47 — Sports Tournament Ranking System

Question / Problem Statement

Store match results and team scores, calculate points and goal differences, and rank teams using a defined competition policy.

Objectives

- Calculate points from match results.
- Track goals scored and conceded.
- Rank teams by points and then goal difference.

Assumptions

- A win earns 3 points, a draw earns 1 point, and a loss earns 0 points.
- Each match is represented by (team1, team2, goals1, goals2).
- Ranking uses points first, then goal difference.

Algorithm

1. Initialize statistics for every team.
2. Process each match and update goals and points.
3. Calculate goal difference for each team.
4. Sort teams by points and goal difference in descending order.
5. Print the tournament table.

Pseudocode

```
START
Initialize each team's points, goals for and goals against
FOR each match: update goals
Award 3 points for win or 1 point each for draw
Calculate goal difference
Sort by points, then goal difference; display ranking
END
```

Python Program

```
teams = ["Falcons", "Tigers", "Eagles"]
matches = [
    ("Falcons", "Tigers", 2, 1),
    ("Eagles", "Falcons", 1, 1),
    ("Tigers", "Eagles", 0, 2)
]
stats = {team: {"points": 0, "for": 0, "against": 0}
         for team in teams}

for a, b, ga, gb in matches:
    stats[a]["for"] += ga
    stats[a]["against"] += gb
    stats[b]["for"] += gb
    stats[b]["against"] += ga
    if ga > gb:
        stats[a]["points"] += 3
    elif gb > ga:
        stats[b]["points"] += 3
    else:
        stats[a]["points"] += 1
        stats[b]["points"] += 1

ranking = sorted(
    teams,
    key=lambda t: (stats[t]["points"],
                   stats[t]["for"] - stats[t]["against"]),
    reverse=True
)
for position, team in enumerate(ranking, start=1):
    s = stats[team]
    gd = s["for"] - s["against"]
    print(position, team, "Points:", s["points"], "GD:", gd)
```

Sample Output



Test Cases

Input / Condition	Expected Result
Win	Winner receives 3 points
Draw	Both teams receive 1 point
Equal points	Goal difference breaks the tie

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The ranking system processes match results and applies points and goal-difference rules to generate a consistent table.

CASE STUDY 33 — Campus Registration Identifier Analyzer

Question / Problem Statement

Validate student registration identifiers, extract their components, and classify records using branch and admission-year codes.

Objectives

- Check the structure of registration identifiers.
- Extract admission year and branch code.
- Report invalid identifiers clearly.

Assumptions

- The example format is YYBRNNNN, eight characters total.
- YY contains two digits, BR is a two-letter branch code, and NNNN contains four digits.
- Accepted branch codes are CSD, CSE, and ECE in this sample only; because the format has two branch characters, accepted codes used here are CS and EC.

Algorithm

1. Store sample registration IDs.
2. Check length and character positions.
3. Extract year, branch, and sequence if valid.
4. Classify the branch and print the result.

Pseudocode

```
START
Set sample registration identifiers and allowed branch codes
FOR each ID: check length and required character types
IF valid: extract year, branch and sequence
Print valid details or an invalid-format message
END
```

Python Program

```
ids = ["26CS0141", "25EC0023", "26CSD141", "ABCS1234"]
valid_branches = ["CS", "EC"]

for student_id in ids:
    valid = (
        len(student_id) == 8
        and student_id[:2].isdigit()
        and student_id[2:4].isalpha()
        and student_id[4:].isdigit()
        and student_id[2:4] in valid_branches
    )
    if valid:
        year = "20" + student_id[:2]
        branch = student_id[2:4]
        sequence = student_id[4:]
        print(student_id, "- Valid; year:", year,
              "branch:", branch, "sequence:", sequence)
    else:
        print(student_id, "- Invalid identifier")
```

Sample Output



Test Cases

Input / Condition	Expected Result
26CS0141	Valid eight-character ID
26CSD141	Invalid length
ABCS1234	Invalid year digits

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The analyzer checks identifier structure before extracting fields, reducing errors when processing student registration records.

CASE STUDY 64 — Tuple-Based Course Registration Validation

Question / Problem Statement

Represent course registrations as tuples, detect duplicate registrations and invalid course combinations, and report students whose selections violate the supplied rules.

Objectives

- Store registrations using tuples.
- Detect duplicate student-course entries.
- Check course combinations against simple prerequisites.

Assumptions

- Each tuple contains (student_id, course_code).
- A student may register for a course only once.
- AI201 requires CS101 to be registered by the same student.

Algorithm

1. Store registration tuples.
2. Track each student-course pair to find duplicates.
3. Build a set of courses for each student.
4. Check the prerequisite rule for AI201.
5. Display duplicate and prerequisite errors.

Pseudocode

```
START
Read registration tuples
Track seen pairs and report duplicates
Build course list for each student
Check whether AI201 students also have CS101
Display validation results
END
```

Python Program

```
registrations = [
    ("S101", "CS101"),
    ("S101", "AI201"),
    ("S102", "AI201"),
    ("S101", "CS101"),
    ("S102", "CS102")
]
seen = []
student_courses = {}

for student, course in registrations:
    pair = (student, course)
    if pair in seen:
        print("Duplicate registration:", pair)
    else:
        seen.append(pair)
    if student not in student_courses:
        student_courses[student] = []
    if course not in student_courses[student]:
        student_courses[student].append(course)

for student, courses in student_courses.items():
    if "AI201" in courses and "CS101" not in courses:
        print(student, "- Invalid: AI201 requires CS101")
    else:
        print(student, "- Course selection valid")
```

Sample Output



Test Cases

Input / Condition	Expected Result
Same student and course twice	Duplicate detected

Input / Condition	Expected Result
AI201 without CS101	Prerequisite error
AI201 with CS101	Selection valid

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

Tuple-based records and list checks help identify duplicate registrations and prerequisite violations.

CASE STUDY 75 — Integrated Student Record Consistency Checker

Question / Problem Statement

Validate student identifiers, names, marks, attendance, and course records; detect duplicate IDs, missing values, and inconsistent academic records.

Objectives

- Check required student fields.
- Validate marks and attendance ranges.
- Detect duplicate IDs and report record problems.

Assumptions

- Each record is (ID, name, marks, attendance, course).
- Marks must be 0–100 and attendance must be 0–100.
- ID, name, and course must not be empty.

Algorithm

1. Store student records as tuples.
2. Track previously seen identifiers.
3. Validate required fields and numeric ranges.
4. Print each record's validation status.
5. Summarize records with issues.

Pseudocode

```
START
Read student record tuples
FOR each record: detect duplicate ID
Check required fields and mark/attendance ranges
Print each detected issue
Display record status
END
```

Python Program

```
records = [
    ("S101", "Anu", 86, 92, "Python"),
    ("S102", "Bala", 105, 80, "Python"),
    ("S101", "Chandu", 74, 88, "C"),
    ("S104", "", 67, 105, "Python")
]
seen_ids = []

for sid, name, marks, attendance, course in records:
    issues = []
    if sid in seen_ids:
        issues.append("duplicate ID")
    else:
        seen_ids.append(sid)
    if not sid or not name.strip() or not course.strip():
        issues.append("missing required field")
    if not 0 <= marks <= 100:
        issues.append("marks out of range")
    if not 0 <= attendance <= 100:
        issues.append("attendance out of range")

    if issues:
        print(sid, "- CHECK:", ", ".join(issues))
    else:
        print(sid, "- Valid record")
```

Sample Output



Test Cases

Input / Condition	Expected Result
Marks 105	Out-of-range issue
Repeated S101	Duplicate ID issue
Blank name	Missing field issue

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The checker applies consistent validation rules to student data and identifies common record-quality problems.

CASE STUDY 89 — University Hostel Room Allocation

Question / Problem Statement

Allocate students to hostel rooms according to room capacity, detect duplicate assignments, and calculate how many students remain unallocated.

Objectives

- Allocate students without exceeding room capacity.
- Prevent assigning the same student twice.
- Report occupied and vacant beds and unallocated students.

Assumptions

- Rooms are represented by a dictionary of room capacities.
- Students are allocated in the order listed.
- Each student needs one bed and may be assigned only once.

Algorithm

1. Set room capacities and student list.
2. Create an empty allocation list for each room.
3. For each student, find the first room with space.
4. Allocate the student or mark them unallocated.
5. Display room assignments and remaining demand.

Pseudocode

```
START
Set room capacities and student list
Create empty room lists and assigned-student list
FOR each student: skip duplicate; otherwise find room with space
Allocate or add to unallocated list
Display room occupancy and unallocated students
END
```

Python Program

```
room_capacity = {"A101": 2, "A102": 1, "B201": 2}
students = ["S101", "S102", "S103", "S104", "S102", "S105"]
rooms = {room: [] for room in room_capacity}
assigned = []
unallocated = []

for student in students:
    if student in assigned:
        print(student, "- duplicate request ignored")
        continue
    placed = False
    for room, capacity in room_capacity.items():
        if len(rooms[room]) < capacity:
            rooms[room].append(student)
            assigned.append(student)
            placed = True
            print(student, "allocated to", room)
            break
    if not placed:
        unallocated.append(student)
        print(student, "- no bed available")

for room, occupants in rooms.items():
    print(room, "occupants:", occupants,
          "vacant beds:", room_capacity[room] - len(occupants))
print("Unallocated students:", unallocated)
```

Sample Output

Python Console

```
S101 allocated to A101
S102 allocated to A101
S103 allocated to A102
S104 allocated to B201
S102 - duplicate request ignored
S105 allocated to B201
A101 occupants: ['S101', 'S102'] vacant beds: 0
A102 occupants: ['S103'] vacant beds: 0
B201 occupants: ['S104', 'S105'] vacant beds: 0
Unallocated students: []
```

Test Cases

Input / Condition	Expected Result
Room has free bed	Student is allocated
Student appears twice	Second request ignored
All beds full	Next student remains unallocated

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The allocation program respects room capacities, avoids duplicate assignments, and reports occupancy and unallocated demand.

CASE STUDY 92 — Multi-Event Sports Scheduling and Scoring

Question / Problem Statement

Represent event schedules and team scores using lists and tuples, detect overlapping events for the same team, total team points, and generate final rankings.

Objectives

- Store event schedules and results in tuples.
- Detect schedule conflicts for teams.
- Aggregate points and rank teams.

Assumptions

- Each schedule tuple is (event, team, start_hour, end_hour).
- Two events conflict when they involve the same team and their time intervals overlap.
- Results are represented by (team, points); rankings use total points.

Algorithm

1. Store schedule tuples and inspect pairs for same-team time overlap.
2. Print any schedule conflicts.
3. Add points from all event results for each team.
4. Sort teams by total points in descending order.
5. Display conflicts and final ranking.

Pseudocode

```
START
Read event schedules
Compare schedule pairs for same-team overlap
Read event points and total points by team
Sort teams by total points
Display conflicts and ranking
END
```

Python Program

```
schedule = [
    ("Basketball", "Falcons", 9, 11),
    ("Volleyball", "Falcons", 10, 12),
    ("Chess", "Tigers", 9, 10),
    ("Relay", "Tigers", 10, 11)
]
results = [
    ("Falcons", 5), ("Tigers", 3),
    ("Falcons", 2), ("Tigers", 4),
    ("Eagles", 6)
]

conflicts = 0
for i in range(len(schedule)):
    event1, team1, start1, end1 = schedule[i]
    for j in range(i + 1, len(schedule)):
        event2, team2, start2, end2 = schedule[j]
        if team1 == team2 and start1 < end2 and start2 < end1:
            print("Conflict:", team1, event1, "and", event2)
            conflicts += 1

totals = {}
for team, points in results:
    totals[team] = totals.get(team, 0) + points

ranking = sorted(totals.items(),
                 key=lambda item: item[1], reverse=True)
for position, (team, points) in enumerate(ranking, start=1):
    print(position, team, "- points:", points)
print("Total schedule conflicts:", conflicts)
```

Sample Output

Python Console

```
Conflict: Falcons Basketball and Volleyball
1 Falcons - points: 7
2 Tigers - points: 7
3 Eagles - points: 6
Total schedule conflicts: 1
```

Test Cases

Input / Condition	Expected Result
Same team, overlapping time	Conflict detected
Different teams, same time	No team conflict
Multiple results for one team	Points are added

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The program checks team scheduling overlaps and totals event points. Rankings use total points; tied teams may need an additional tie-break rule if required.